

Q1. What are Angular ?

Angular is a front-end framework developed by Google. It is used to build dynamic and single page application (SPA) using typescript

Angular follows component based architecture

Q2. What are Angular advantages?

- Component-based architecture
- Two-way data binding
- Dependency Injection
- Routing support
- Fast development
- Easy testing
- Reusable code

Q3. Difference between AngularJS and Angular?

AngularJS	Angular
Based on JavaScript	Based on TypeScript
MVC Architecture	Component-Based Architecture
Slower	Faster
Uses Controllers	Uses Components
Uses \$scope	Uses Data Binding
Mobile support is limited	Better mobile support
No Lazy Loading	Supports Lazy Loading

Q4. What is NPM?

NPM (Node Package Manager) is used to **install and manage Angular packages and dependencies.**

Example:

```
npm install
npm install @angular/material
npm update
npm uninstall package-name
```

Q5. What is Angular CLI?

Angular CLI (Command Line Interface) is a tool used to create, build, test, and run Angular applications.

Common Commands

Create project

`ng new project-name`

Run application

`ng serve`

Generate component

`ng generate component home`

Generate service

`ng generate service employee`

Build application

`ng build`

Run tests

`ng test`

Q6. What are Components in Angular?

Answer:

A Component is the basic building block of an Angular application. It controls a part of the UI and contains:

- HTML Template
- TypeScript Class
- CSS Styles

Example:

```
@Component({
  selector:'app-home',
  templateUrl:'home.component.html'
})
export class HomeComponent{}
```

Q7. What is Selector and Template?

Selector is used to uniquely identify a component

The selector is the HTML tag used to display a component.

Example:

```
selector:'app-home'
```

Usage:

```
<app-home></app-home>
```

Template

The template contains the HTML code of the component.

Example:

```
template:`<h1>Welcome</h1>`
```

Q8. What is Module in Angular? What is app.module.ts?

Answer:

A Module groups related components, directives, pipes, and services.

app.module.ts is the root module of an Angular application.

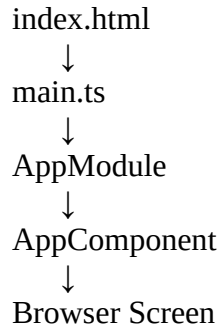
It contains:

- Declarations
- Imports
- Providers
- Bootstrap component

Q9. How does an Angular application start?

Answer:

Flow:



Explanation:

- **index.html** contains `<app-root>`
- **main.ts** bootstraps `AppModule`
- **AppModule** loads `AppComponent`
- **AppComponent** renders inside `<app-root>`

Q10. What is Bootstrapped Module and Bootstrapped Component?

Bootstrapped Module

The first module loaded by Angular.

Example:

`AppModule`

Bootstrapped Component

The first component displayed.

Example:

`bootstrap:[AppComponent]`

Q11. What is Data Binding in Angular?

Data binding is used to connect data between the component and the template.

Types of data binding:

- Interpolation
- Property Binding
- Event Binding

- Two-way Binding

Q12. What is String Interpolation?

Answer:

Interpolation is used to display data from the component to the HTML using double curly braces `{{ }}`.

Data Flow

Component → View

Example:

app.component.ts

```
name = "Jyothi";
```

app.component.html

```
<h1>{{name}}</h1>
```

Output

Jyothi

Q13. What is Property Binding?

Property binding is used to send data from component to HTML element property using `[]`

Data Flow

Component → View

Example

In ts

```
imageUrl="logo.png";
```

Html

```
<img [src]="imageUrl">
```

Q14. What is Event Binding?

Event binding is used to handle user actions like click events.

Data Flow

View → Component

Example

```
<button (click)="save()">Save</button>
```

```
save(){  
  console.log("Saved");  
}
```

Q15. What is Two-Way Data Binding?

Two-way data binding allows data transfer in both directions:

- Component to view
- View to component

Two-Way Data Binding combines Property Binding and Event Binding.

It uses:

```
[(ngModel)]
```

Example

```
<input [(ngModel)]="name">
```

```
<h2>{{ name }}</h2>
```

If the user types

Angular

Output

Angular

Data Flow

Component ↔ View

Note:

Import **FormsModule** to use ngModel.

Q16. What are Directives? What are the types?

Directives are used to modify the structure or behavior of DOM elements.

Types:

- Structural directives
- Attribute directives
- Component directives

What are Structural Directives?

Structural directives change DOM structure.

Examples

- : • *ngIf
- *ngFor
- *ngSwitch

What are Attribute Directives?

Attribute directives change appearance or behavior of elements.

Examples:

- ngClass
- ngStyle

What is *ngIf?

*ngIf is a structural directive used for condition checking.

Example

```
<p *ngIf="isLoggedIn">
Welcome User
</p>
```

If

isLoggedIn=true;

Output

Welcome User

If false, nothing is displayed.

What is *ngFor?

*ngFor is used to iterate over a collection

```
students=["Ram","John","Sam"];
```

```
<li *ngFor="let s of students">
  {{s}}
</li>
```

Output

Ram
John
Sam

Q19. What is *ngSwitch?

Answer:

*ngSwitch is used to display one element from multiple options based on a condition.

Example

```
<div [ngSwitch]="role">
```

```
<p *ngSwitchCase="'Admin'">
  Admin Page
</p>
```

```
<p *ngSwitchCase="'User'">
  User Page
</p>
```

```
<p *ngSwitchDefault>
  Guest
</p>
```

```
</div>
```

Q20. What is ngStyle?

Answer:

ngStyle is an Attribute Directive used to apply CSS styles dynamically.

Example

```
<p [ngStyle]='{'color':'red'}'>  
Angular  
</p>
```

Or

```
color="blue";
```

```
<p [ngStyle]='{'color':color}'>  
Hello  
</p>
```

Q21. What is ngClass?

Answer:

ngClass is an **Attribute Directive** used to add or remove CSS classes dynamically based on a condition.

Example:

```
isActive = true;
```

```
<p [ngClass]='{'active': isActive}'>  
Angular  
</p>
```

If isActive = true, the **active** CSS class is applied.

Q23. What is a Decorator?

Decorators provides metadata to classes and define the behavior of classes such as components, services, and modules.”

“Decorators tell Angular how a class should behave.”

Q24. What are the types of Decorators?

Answer:

1. Class Decorators

Used on classes.

Examples:

- @Component
- @NgModule
- @Injectable
- @Directive
- @Pipe

2. Property Decorators

Used on variables.

Examples:

- @Input
- @Output

3. Method Decorators

Used on methods.

Example:

- @HostListener

4. Parameter Decorators

Used on constructor parameters.

Example:

- @Inject

Q25. What are Pipes? Types of Pipes? Parameterized Pipes?

What is a Pipe?

A Pipe is used to transform data before displaying it in the UI.

Example:

{{name | uppercase}}

Output:

JYOTHI

Built-in Pipes

- UpperCase
- LowerCase
- Date
- Currency
- Percent
- Decimal
- Json
- Slice

Parameterized Pipe

A pipe that accepts parameters.

Example:

```
{{today | date:'dd/MM/yyyy'}}
```

Output:

05/07/2026

Another example:

```
{{salary | currency:'INR'}}
```

Output:

₹50,000

Custom pipe

“Custom pipe is used to create our own data transformation in Angular.”

“First, we create a pipe using Angular CLI: `ng g p pipeName`

“Then the pipe class implements `PipeTransform` interface and we write logic inside the `transform()` method.

Example:

```
export class DemoPipe implements PipeTransform {  
  transform(value: string): string {  
    return value.toUpperCase();  
  }  
}
```

“Finally, we use the pipe in HTML using `| symbol`.” Example: `{{name | demo}}`

Q26. What is Chaining Pipes?

Answer:

Applying multiple pipes to the same value is called Pipe Chaining.

Example:

```
{{today | date:'dd/MM/yyyy' | uppercase}}
```

Here, Angular first formats the date and then applies the uppercase pipe.

Q27. Explain Services with an Example.

What is a Service?

A Service is a class that contains reusable business logic that can be shared across multiple components.

Examples:

- API calls
- Database operations
- Authentication
- Logging
- Validation

Example

```
@Injectable({  
  providedIn:'root'  
})
```

```
export class EmployeeService{
```

```
  getEmployees(){  
    return ["John","Ram","David"];  
  }
```

```
}
```

Using the service:

```
constructor(private empService:EmployeeService){}
```

```
employees=this.empService.getEmployees();
```

Q28. How do you create a Service in Angular?

Using Angular CLI:

```
ng generate service employee
```

or

```
ng g s employee
```

Files created:

```
employee.service.ts
```

```
employee.service.spec.ts
```

Dependency Injection

Dependency injection is used to provide required services or objects to the component without creating them manually

Dependency injection is a design pattern in angular

Q29. How do you use Dependency Injection with Services?

Answer:

Inject the service through the constructor.

```
constructor(private employeeService:EmployeeService){}
```

Now you can call:

```
this.employeeService.getEmployees();
```

Q30.What is Hierarchical Dependency Injection?

Hierarchical Dependency Injection is the mechanism in which services are provided at different levels of application, such as root level, module level, or component level.

A service can be provided at different levels:

- Root Level (providedIn: 'root') → Singleton across the application.
- Module Level → Shared within the module.
- Component Level → New instance for each component.

Angular follows a hierarchy to find and inject the required service. If the service is not found at current level, it searches the parent level until it finds it.

Q31. What is a Provider in Angular?

A provider is responsible for creating and supplying service to a component or another service through dependency injection

A **Provider** tells Angular **how to create and supply an instance of a service**.

Example:

```
@Injectable({  
  providedIn: 'root'  
})  
export class EmployeeService { }
```

Or in a component:

```
@Component({  
  providers: [EmployeeService]  
})
```

Q32. What is the role of @Injectable Decorator?

Answer:

@Injectable marks a class as a service that can participate in Dependency Injection.

Example:

```
@Injectable({  
  providedIn: 'root'  
})  
  
export class EmployeeService {  
  
}
```

Q33. What are Parent and Child Components?

Parent component is the component that contains another component

Child component is the component that is placed inside the parent component

The parent component send the data to child component using @Input() and the child component can send back to the parent component using @Output and EventEmitter

EventEmitter

EventEmitter is used to send data or events from a child component to a parent component using the @Output decorator

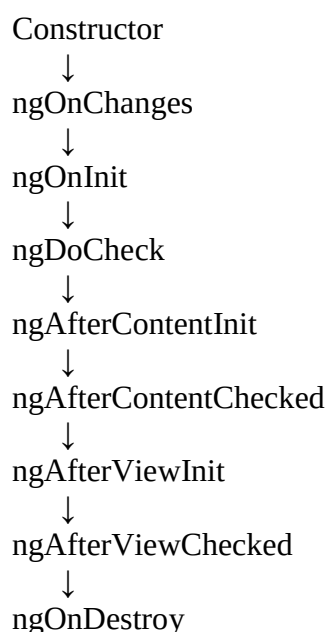
Q34. What are Lifecycle Hooks in Angular?

LifeCycle hooks are Angular methods that are automatically called at different stages of component's lifecycle to perform specific tasks

Common Lifecycle Hooks

- ☐ **ngOnChanges()** → Called when an @Input() value changes.
- ☐ **ngOnInit()** → Called once after component initialization (API calls, data loading).
- ☐ **ngDoCheck()** → Detects custom changes.
- ☐ **ngAfterContentInit()** → Called after external content (ng-content) is initialized.
- ☐ **ngAfterContentChecked()** → Called after Angular checks projected content.
- ☐ **ngAfterViewInit()** → Called after the component's view and child views are initialized.
- ☐ **ngAfterViewChecked()** → Called after Angular checks the component's view.
- ☐ **ngOnDestroy()** → **Called before the component is destroyed (cleanup, unsubscribe).**

Lifecycle Flow



Q35. What is Constructor in Angular?

A constructor is a special method that is automatically called when the component is created, and is mainly used for dependency injection

Q36. What is ngOnInit()?

“ngOnInit is a life cycle hook that automatically called after the component is initialized. It is mainly used to perform initialization tasks such as calling APIs , loading data and setting default values.

Q37. Difference between Constructor and ngOnInit()

Constructor	ngOnInit()
It is a TypeScript feature .	It is an Angular Lifecycle Hook .
Used for Dependency Injection .	Used for initialization logic .
Called when the component is created .	Called after Angular initializes the component .
Used to inject services .	Used for API calls, loading data, and setting default values .

Q38. What are Asynchronous Operations?

Asynchronous operations are the operations that run in the background without blocking application

Examples:

- HTTP API Calls
- File Upload
- Database Calls
- Timers
- User Events

Angular handles asynchronous operations using promises and Observables(RxJS)

Q39. Difference between Promise and Observable?

Promise	Observable
Emits one value	Emits multiple values
Cannot be cancelled	Can be cancelled
Not lazy	Lazy (executes on subscribe)
Uses .then()	Uses .subscribe()
Native JavaScript	RxJS

“A promise returns only one value and cannot be cancelled once it starts. It uses then() and catch() to handle the result. An observable can return multiple values over time and can be cancelled using unsubscribe(). It uses subscribe() to receive data. In angular Observables are preferred because HTTPClient and many features use observables.

Q40. What is RxJS?

RxJS stands for **Reactive Extensions for JavaScript**.

It is a library used for handling asynchronous programming using **Observables**.

Common Operators

- map()
- filter()
- tap()

Q41. What is an Observable? How do you implement an Observable?

Answer:

An **Observable** is an RxJS object that emits data over time. It is widely used for handling asynchronous operations such as API calls, events, and timers.

Steps to use an Observable:

1. Create or receive an Observable.
2. Subscribe to it.
3. Process the emitted data.

Example:

```
// employee.service.ts
```

```
getEmployees() {
```

```

    return this.http.get('https://api.example.com/employees');
  }

// employee.component.ts

constructor(private empService: EmployeeService) {}

ngOnInit() {
  this.empService.getEmployees().subscribe(
    data => {
      console.log(data);
    },
    error => {
      console.log(error);
    }
  );
}

```

Q42. What is the role of HttpClient in Angular?

Answer:

HttpClient is a service used to communicate with backend APIs.

It returns observables

It supports:

- GET
- POST
- PUT
- DELETE
- PATCH

First import **HttpClientModule**.

```

imports: [
  HttpClientModule
]

```

Inject HttpClient:

```

constructor(private http: HttpClient){}

```

Example:

```

getUsers(){
return this.http.get("https://api.example.com/users");
}

```

Q43. What is TypeScript?

TypeScript is a **superset of JavaScript** developed by Microsoft.

It adds:

- Static Typing
- Interfaces
- Classes
- Generics
- Better IDE support

Difference

JavaScript	TypeScript
Js is a scripting language	TS is superset of JS
Dynamically typed	Statically typed
No compile-time checking	Compile-time error checking
Less secure	More secure
No interfaces	Supports interfaces
Easier to write	Easier to maintain

Q44. Difference between var, let and const?

var	let	const
Function scope	Block scope	Block scope
Can redeclare	Cannot redeclare	Cannot redeclare
Can reassign	Can reassign	Cannot reassign
Hoisted	Hoisted (TDZ)	Hoisted (TDZ)

Q45. What is Type Annotation?

Answer:

Type Annotation specifies the data type of a variable, parameter, or return value.

Example:

```
let name:string="Jyothi";
```

```
let age:number=22;
```

```
let isActive:boolean=true;
```

Q46. What are Built-in (Primitive) and User-Defined (Non-Primitive) Types in TypeScript?

Primitive Types

- string
- number
- boolean
- null
- undefined
- bigint
- symbol

Example:

```
let name:string="John";
```

```
let age:number=25;
```

User-Defined (Non-Primitive) Types

- Class
- Interface
- Enum
- Array
- Tuple

Example:

```
interface Employee{  
  id:number;  
  name:string;  
}
```

Q47. What is any type?

Answer:

any allows a variable to store any type of value.

Example:

```
let data:any;
```

Q48. What is Enum?

An **Enum** is a user-defined data type used to define a set of named constants.

Example:

```
enum Role{  
  Admin,  
  User,  
  Manager  
}
```

```
let role=Role.Admin;
```

Q49. What is Type Assertion?

Type Assertion tells the TypeScript compiler to treat a value as a specific type.

We use Type Assertion to avoid type-related errors and to access the properties and methods of the correct data type

Example:

```
let value:any="Angular";
```

```
let length=(value as string).length;
```

Q50. What are Arrow Functions?

Answer:

Arrow functions are a shorter way to write functions and automatically bind this.

Example:

```
let add=(a:number,b:number)=>{  
  return a+b;  
}
```

. What is Routing?

Routing is used to navigate between different components or pages in an Angular application without reloading the browser.

Angular uses the RouterModule to configure routes.

What is Lazy Loading?

Lazy Loading means modules are loaded only when there is a need

This improves application performance and reduces the initial loading time.

What are @Input() and @Output()?

@Input() is used to pass data from a parent component to a child component. @Output() along with EventEmitter is used to send data from a child component to a parent component.

What are Guards?

Guards are used to protect routes from unauthorized users

. Examples:

- CanActivate
- AuthGuard

HttpInterceptor

HttpInterceptor is used to intercept and modify HTTP requests and responses before they reach the server or the application.

It is mainly used to add authentication tokens, log requests, modify headers, and handle errors globally.

Flow

```
Component
  ↓
HttpInterceptor
  ↓
Backend API
  ↓
HttpInterceptor
  ↓
Component
```

For example, if every API request requires a JWT token , instead of adding the token in each API call, we use an interceptor. The interceptor automatically adds the token to every outgoing request.

Q. What are Forms in Angular?

Forms in Angular are used to collect user input, validate the data, and submit it to the server. Angular provides two types of forms: Template-Driven Forms and Reactive Forms.

Keywords: Collect → Validate → Submit

Q. What are the types of Forms?

There are **2 types of forms**:

1. **Template-Driven Forms**
2. **Reactive Forms**

Q. What is a Template-Driven Form?

A Template-Driven Form is a form where most of the logic is written in the HTML template. It uses the ngModel directive and FormsModule. It is suitable for simple and small forms.

Remember:

- HTML
- ngModel
- FormsModule
- Small Forms

Q. What is a Reactive Form?

A Reactive Form is a form where the logic is written in the TypeScript component. It uses FormGroup, FormControl, and ReactiveFormsModule. It is suitable for large and complex forms because it provides better validation and control.

Remember:

- TypeScript
- FormGroup
- FormControl
- ReactiveFormsModule
- Large Forms

Difference between Template-Driven Form and Reactive Form

Template-Driven

Reactive

Logic in HTML Logic in TypeScript

Uses ngModel Uses FormGroup & FormControl

Template-Driven

Reactive

Uses FormsModule Uses ReactiveFormsModule

Small Forms

Large Forms

What are Validators?

Used for form validation.

Examples:

- required
- minlength
- maxlength

What is Signal in Angular?

Signal is a reactive variable that automatically updates the UI when data changes.

Why do we use Signals?

We use Signals for:

- Better performance
- Automatic updates
- Easier state management

How to create a Signal?

```
count = signal(0);
```

How to access Signal value?

```
count()
```

How to update Signal value?

```
count.set(10);
```

OR

```
count.update(value => value + 1);
```

Which Angular version introduced Signals?

Angular 16.